NAME : P SAI VENKAT RAJU SUBJECT : AI ASSISTED CODING

ROLL NO : 2403A510G0 ASSIGNMENT : 6.1

Task Description #1 (Classes – Employee Management)

• Task: Use AI to create an Employee class with attributes (name, id, salary) and a method to calculate yearly salary.

• Instructions:

o Prompt AI to generate the Employee class.

o Analyze the generated code for correctness and structure.

o Ask AI to

Expected Output #1:

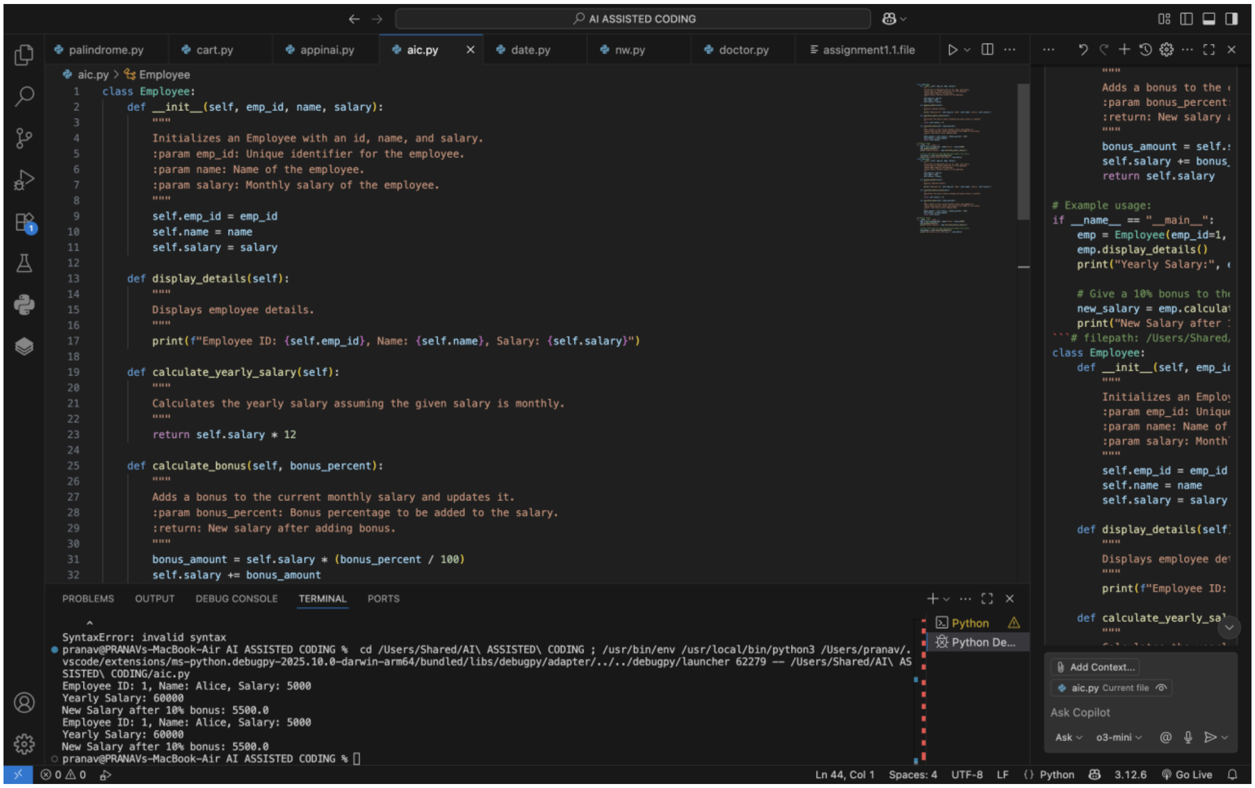
• A class with constructor, display_details(), and calculate_bonus() methods.

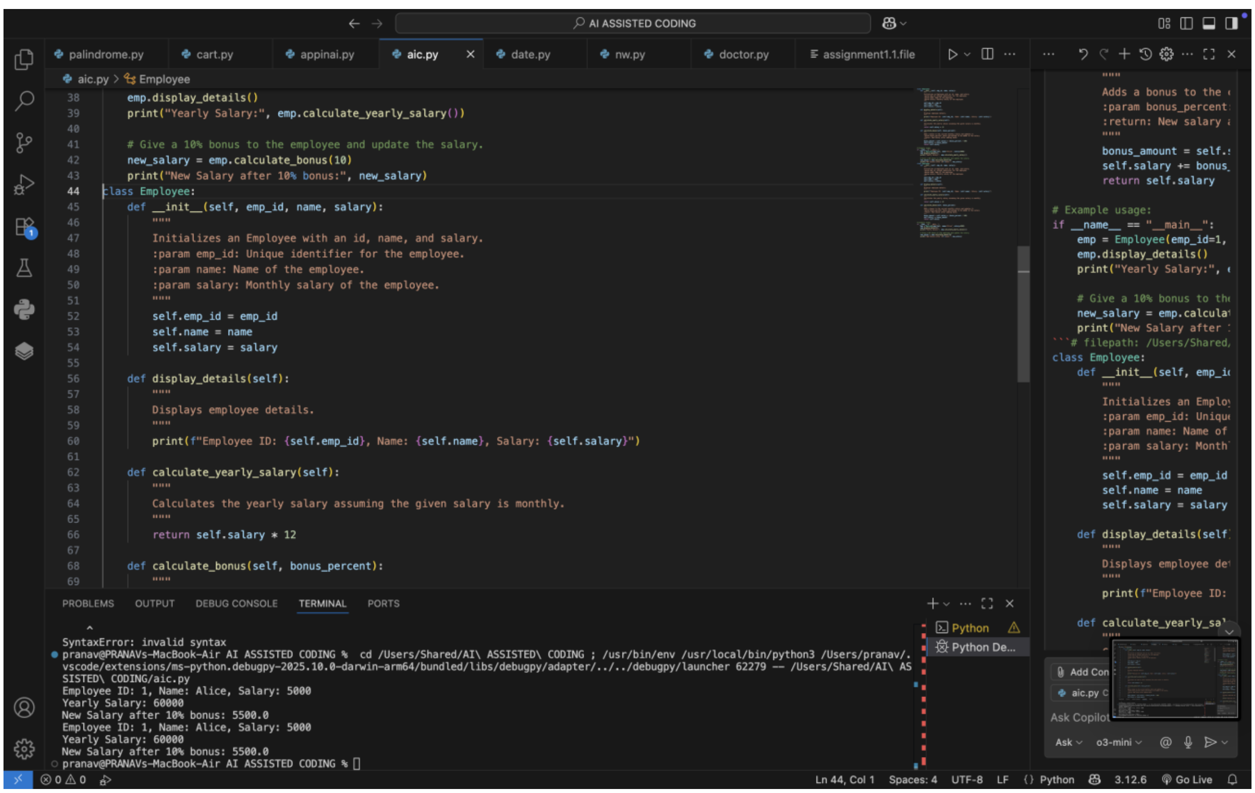
PROMPT :

To create an Employee class with attributes (name, id, salary) and a method to calculate yearly salary.

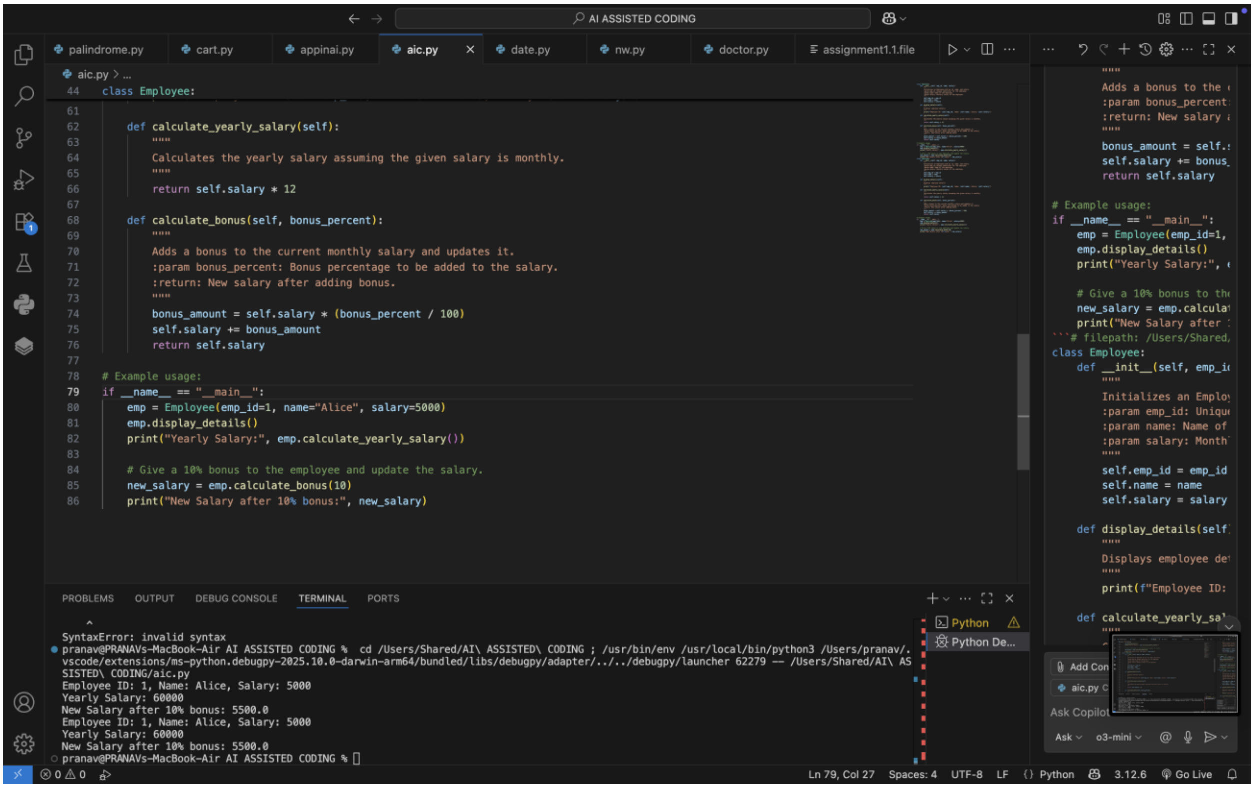
Add a method to give a bonus and recalculate salary.

CODE :


AI ASSISTED CODING
palindrome.py
cart.py
appinai.py
aic.py
date.py
nw.py
doctor.py
assignment1.1.file
aic.py > Employee
class Employee:
    def __init__(self, emp_id, name, salary):
        """
        Initializes an Employee with an id, name, and salary.
        :param emp_id: Unique identifier for the employee.
        :param name: Name of the employee.
        :param salary: Monthly salary of the employee.
        """
        self.emp_id = emp_id
        self.name = name
        self.salary = salary
    def display_details(self):
        """
        Displays employee details.
        """
        print(f"Employee ID: {self.emp_id}, Name: {self.name}, Salary: {self.salary}")
    def calculate_yearly_salary(self):
        """
        Calculates the yearly salary assuming the given salary is monthly.
        """
        return self.salary * 12
    def calculate_bonus(self, bonus_percent):
        """
        Adds a bonus to the current monthly salary and updates it.
        :param bonus_percent: Bonus percentage to be added to the salary.
        :return: New salary after adding bonus.
        """
        bonus_amount = self.salary * (bonus_percent / 100)
        self.salary += bonus_amount
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
SyntaxError: invalid syntax
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING % cd /Users/Shared/AI\ ASSISTED\ CODING ; /usr/bin/env /usr/local/bin/python3 /Users/pranav/.vscode/extensions/ms-python.debugpy-2025.10.0-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 62279 -- /Users/Shared/AI\ ASSISTED\ CODING/aic.py
Employee ID: 1, Name: Alice, Salary: 5000
Yearly Salary: 60000
New Salary after 10% bonus: 5500.0
Employee ID: 1, Name: Alice, Salary: 5000
Yearly Salary: 60000
New Salary after 10% bonus: 5500.0
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Python
Python De...
Add Context...
aic.py Current file
Ask Copilot
Ask
o3-mini
Ln 44, Col 1
Spaces: 4
UTF-8
LF
{} Python
3.12.6
Go Live


AI ASSISTED CODING
palindrome.py
cart.py
appinai.py
aic.py
date.py
nw.py
doctor.py
assignment1.1.file
aic.py > Employee
    emp.display_details()
    print("Yearly Salary:", emp.calculate_yearly_salary())

    # Give a 10% bonus to the employee and update the salary.
    new_salary = emp.calculate_bonus(10)
    print("New Salary after 10% bonus:", new_salary)
class Employee:
    def __init__(self, emp_id, name, salary):
        """
        Initializes an Employee with an id, name, and salary.
        :param emp_id: Unique identifier for the employee.
        :param name: Name of the employee.
        :param salary: Monthly salary of the employee.
        """
        self.emp_id = emp_id
        self.name = name
        self.salary = salary
    def display_details(self):
        """
        Displays employee details.
        """
        print(f"Employee ID: {self.emp_id}, Name: {self.name}, Salary: {self.salary}")
    def calculate_yearly_salary(self):
        """
        Calculates the yearly salary assuming the given salary is monthly.
        """
        return self.salary * 12
    def calculate_bonus(self, bonus_percent):
        """
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
SyntaxError: invalid syntax
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING % cd /Users/Shared/AI\ ASSISTED\ CODING ; /usr/bin/env /usr/local/bin/python3 /Users/pranav/.vscode/extensions/ms-python.debugpy-2025.10.0-darwin-arm64/bundled/libs/debugpy/adapter/../../debugpy/launcher 62279 -- /Users/Shared/AI\ ASSISTED\ CODING/aic.py
Employee ID: 1, Name: Alice, Salary: 5000
Yearly Salary: 60000
New Salary after 10% bonus: 5500.0
Employee ID: 1, Name: Alice, Salary: 5000
Yearly Salary: 60000
New Salary after 10% bonus: 5500.0
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Python
Python De...
Ask Copilot
Ask
o3-mini
Ln 44, Col 1
Spaces: 4
UTF-8
LF
{} Python
3.12.6
Go Live



OUTPUT :

Task Description #2 (Loops – Automorphic Numbers in a Range)

• Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

• Instructions:

o Get AI-generated code to list Automorphic numbers using a for loop.

o Analyze the correctness and efficiency of the generated logic.

o Ask AI to regenerate using a while loop and compare both implementations.

Expected Output #2:

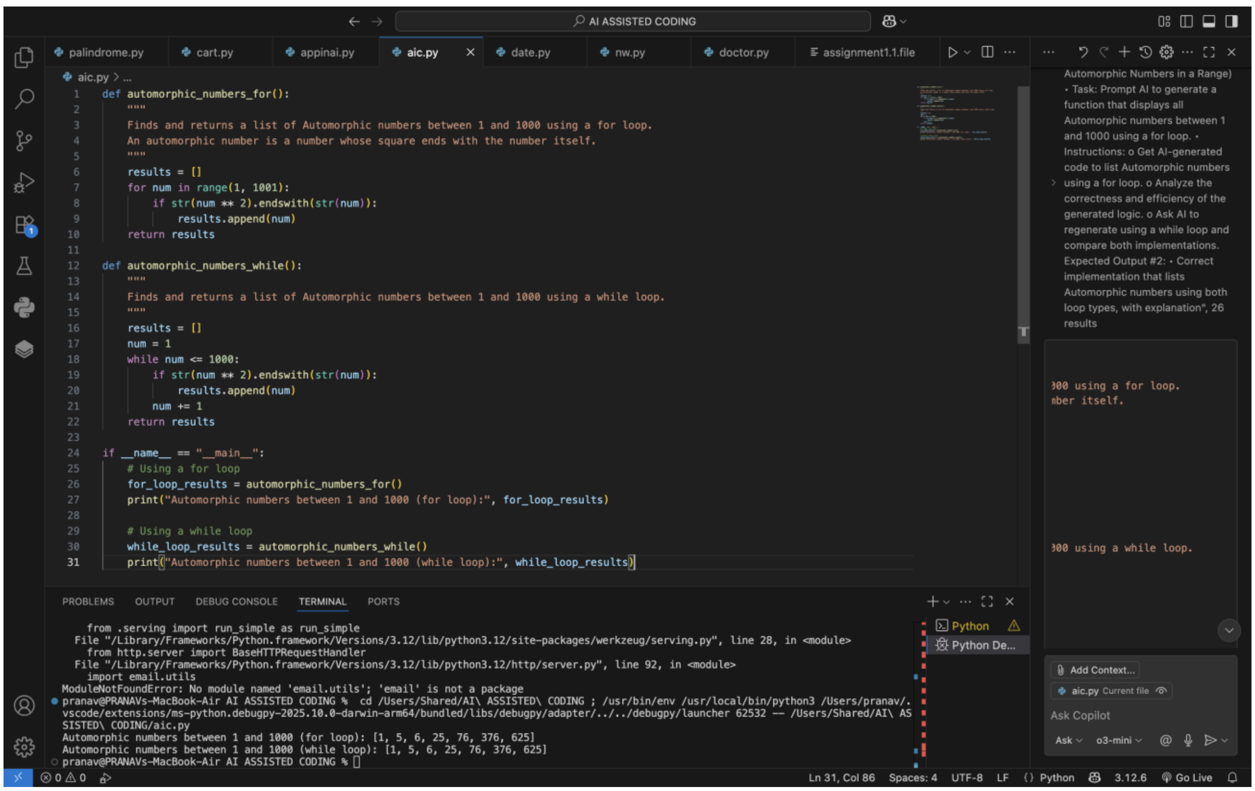
• Correct implementation that lists Automorphic numbers using both loop types, with explanation.

PROMPT :

To generate a function that displays all
Automorphic numbers between 1 and 1000 using a for loop.

To regenerate using a while loop and compare both implementations.

CODE GENERATED :



OUTPUT :

Task Description #3 (Conditional Statements – Online Shopping Feedback Classification)

• Task: Ask AI to write nested if-elif-else conditions to classify online shopping feedback as Positive, Neutral, or Negative based on a numerical rating (1–5).

• Instructions:

o Generate initial code using nested if-elif-else.

o Analyze correctness and readability.

o Ask AI to rewrite using dictionary-based or match-case structure.

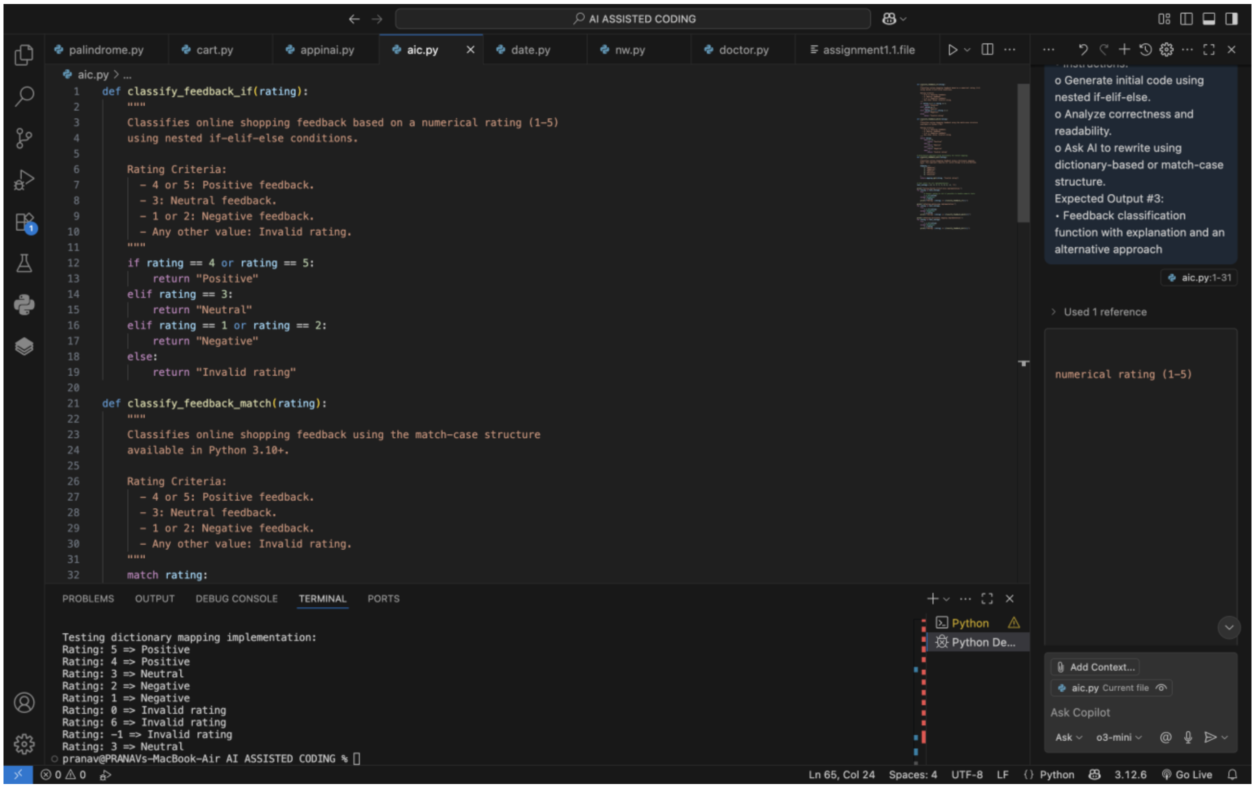
Expected Output #3:

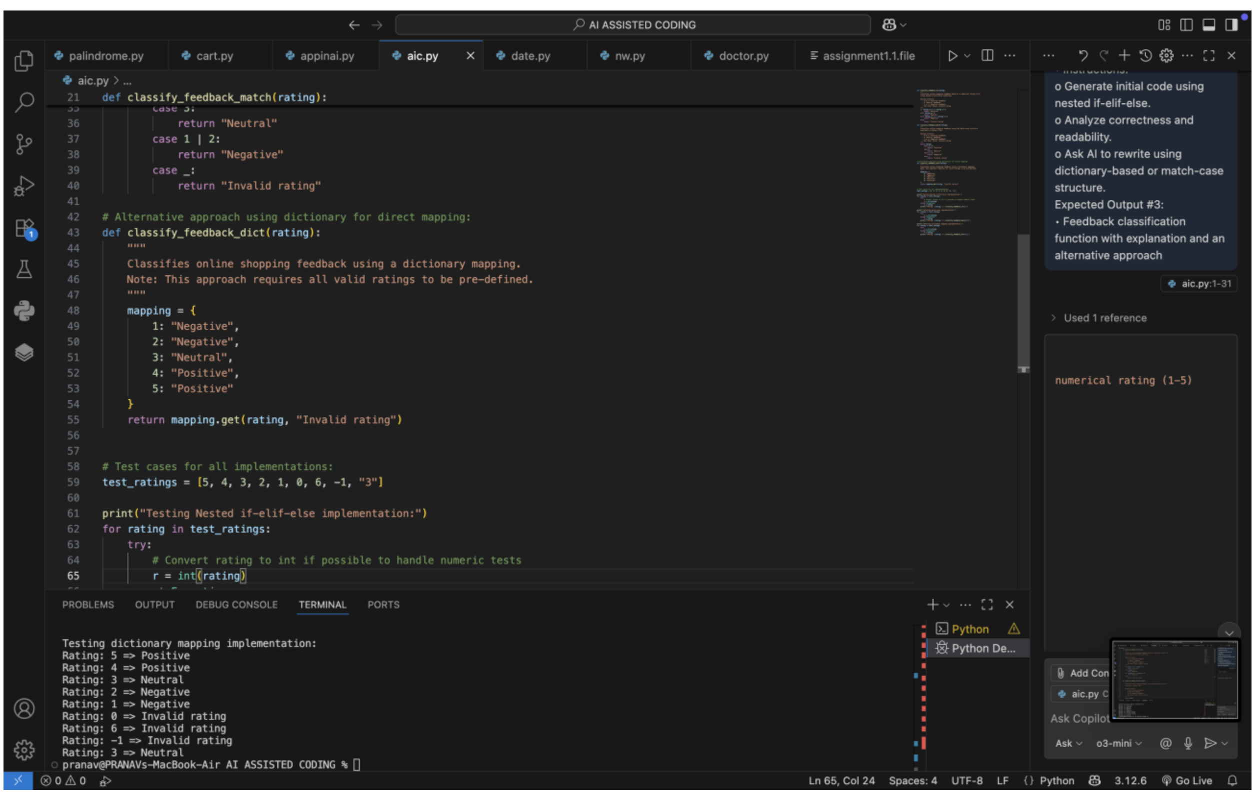
• Feedback classification function with explanation and an alternative approach

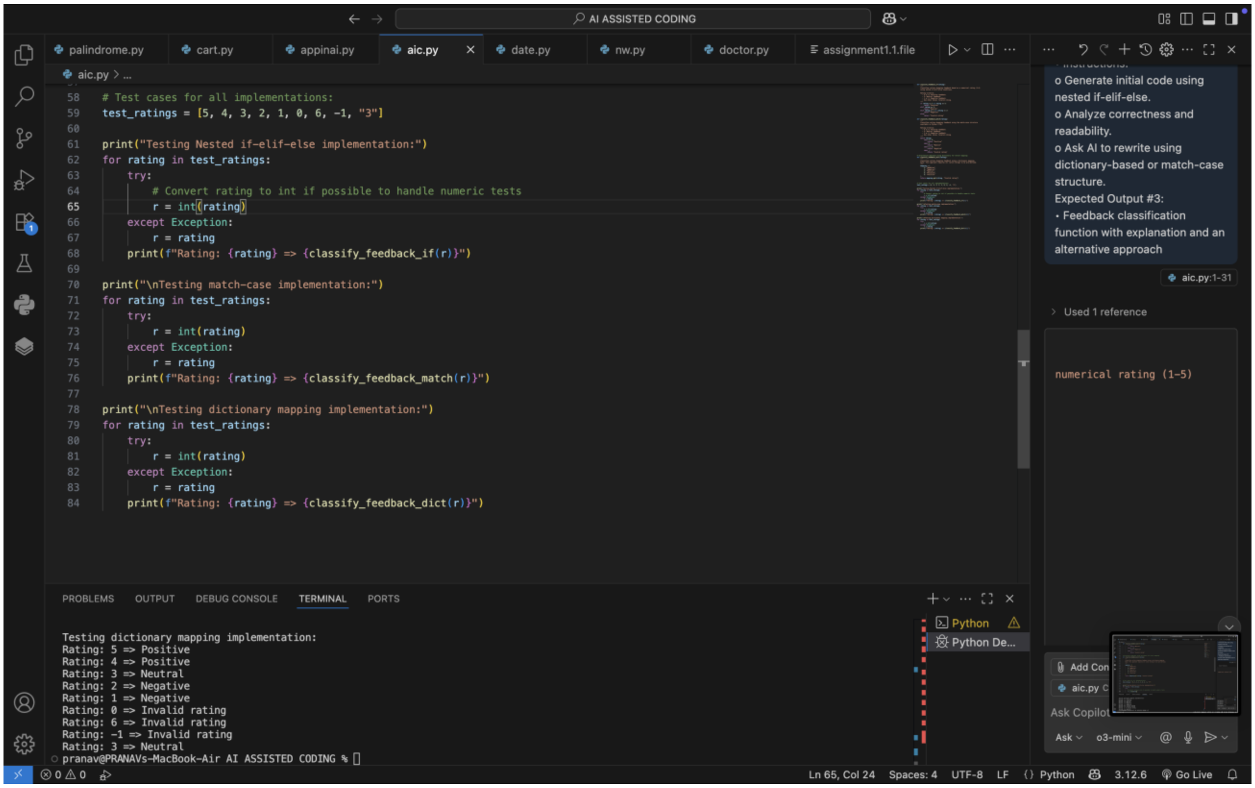
PROMPT :

To write nested if-elif-else conditions to classify
online shopping feedback as Positive, Neutral, or Negative based
on a numerical rating (1–5)

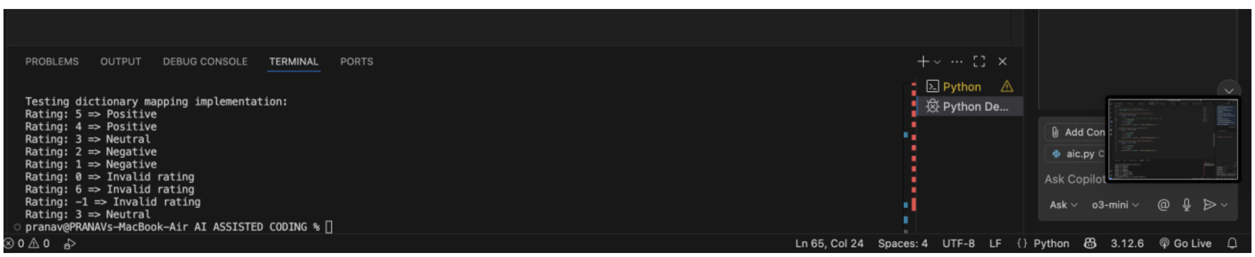
CODE :


AI ASSISTED CODING
palindrome.py
cart.py
appinai.py
aic.py
date.py
nw.py
doctor.py
assignment1.1.file
aic.py > ...
def classify_feedback_if(rating):
"""
Classifies online shopping feedback based on a numerical rating (1-5)
using nested if-elif-else conditions.
Rating Criteria:
- 4 or 5: Positive feedback.
- 3: Neutral feedback.
- 1 or 2: Negative feedback.
- Any other value: Invalid rating.
"""
if rating == 4 or rating == 5:
return "Positive"
elif rating == 3:
return "Neutral"
elif rating == 1 or rating == 2:
return "Negative"
else:
return "Invalid rating"
def classify_feedback_match(rating):
"""
Classifies online shopping feedback using the match-case structure
available in Python 3.10+.
Rating Criteria:
- 4 or 5: Positive feedback.
- 3: Neutral feedback.
- 1 or 2: Negative feedback.
- Any other value: Invalid rating.
"""
match rating:
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Testing dictionary mapping implementation:
Rating: 5 => Positive
Rating: 4 => Positive
Rating: 3 => Neutral
Rating: 2 => Negative
Rating: 1 => Negative
Rating: 0 => Invalid rating
Rating: 6 => Invalid rating
Rating: -1 => Invalid rating
Rating: 3 => Neutral
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Python
Python De...
o Generate initial code using nested if-elif-else.
o Analyze correctness and readability.
o Ask AI to rewrite using dictionary-based or match-case structure.
Expected Output #3:
• Feedback classification function with explanation and an alternative approach
aic.py:1-31
Used 1 reference
numerical rating (1-5)
Add Context...
aic.py Current file
Ask Copilot
Ask
o3-mini
Ln 65, Col 24
Spaces: 4
UTF-8
LF
Python
3.12.6
Go Live


AI ASSISTED CODING
palindrome.py
cart.py
appinai.py
aic.py
date.py
nw.py
doctor.py
assignment1.1.file
aic.py > ...
def classify_feedback_match(rating):
return "Neutral"
case 1 | 2:
return "Negative"
case _:
return "Invalid rating"
# Alternative approach using dictionary for direct mapping:
def classify_feedback_dict(rating):
"""
Classifies online shopping feedback using a dictionary mapping.
Note: This approach requires all valid ratings to be pre-defined.
"""
mapping = {
1: "Negative",
2: "Negative",
3: "Neutral",
4: "Positive",
5: "Positive"
}
return mapping.get(rating, "Invalid rating")
# Test cases for all implementations:
test_ratings = [5, 4, 3, 2, 1, 0, 6, -1, "3"]
print("Testing Nested if-elif-else implementation:")
for rating in test_ratings:
try:
# Convert rating to int if possible to handle numeric tests
r = int(rating)
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Testing dictionary mapping implementation:
Rating: 5 => Positive
Rating: 4 => Positive
Rating: 3 => Neutral
Rating: 2 => Negative
Rating: 1 => Negative
Rating: 0 => Invalid rating
Rating: 6 => Invalid rating
Rating: -1 => Invalid rating
Rating: 3 => Neutral
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Python
Python De...
o Generate initial code using nested if-elif-else.
o Analyze correctness and readability.
o Ask AI to rewrite using dictionary-based or match-case structure.
Expected Output #3:
• Feedback classification function with explanation and an alternative approach
aic.py:1-31
Used 1 reference
numerical rating (1-5)
Ask Copilot
Ask
o3-mini
Ln 65, Col 24
Spaces: 4
UTF-8
LF
Python
3.12.6
Go Live



OUTPUT :



Task Description #4 (Loops – Prime Numbers in a Range)

• Task: Generate a function using AI that displays all prime numbers within a user-specified range (e.g., 1 to 500).

• Instructions:

o Get AI-generated code to list all primes using a for loop.

o Analyze the correctness and efficiency of the prime-checking logic.

o Ask AI to regenerate an optimized version (e.g., using the square root method).

Expected Output #4:

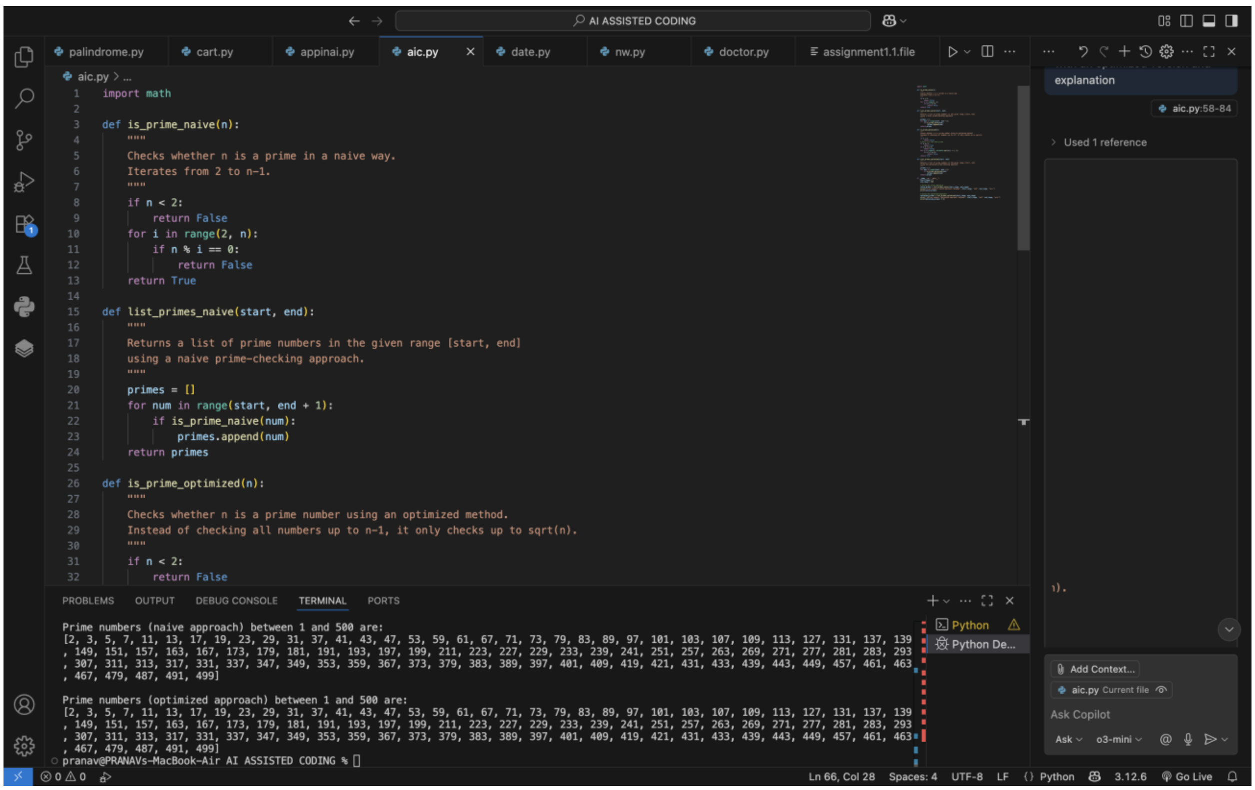
• Python program that lists all prime numbers within a given range, with an optimized version and explanation

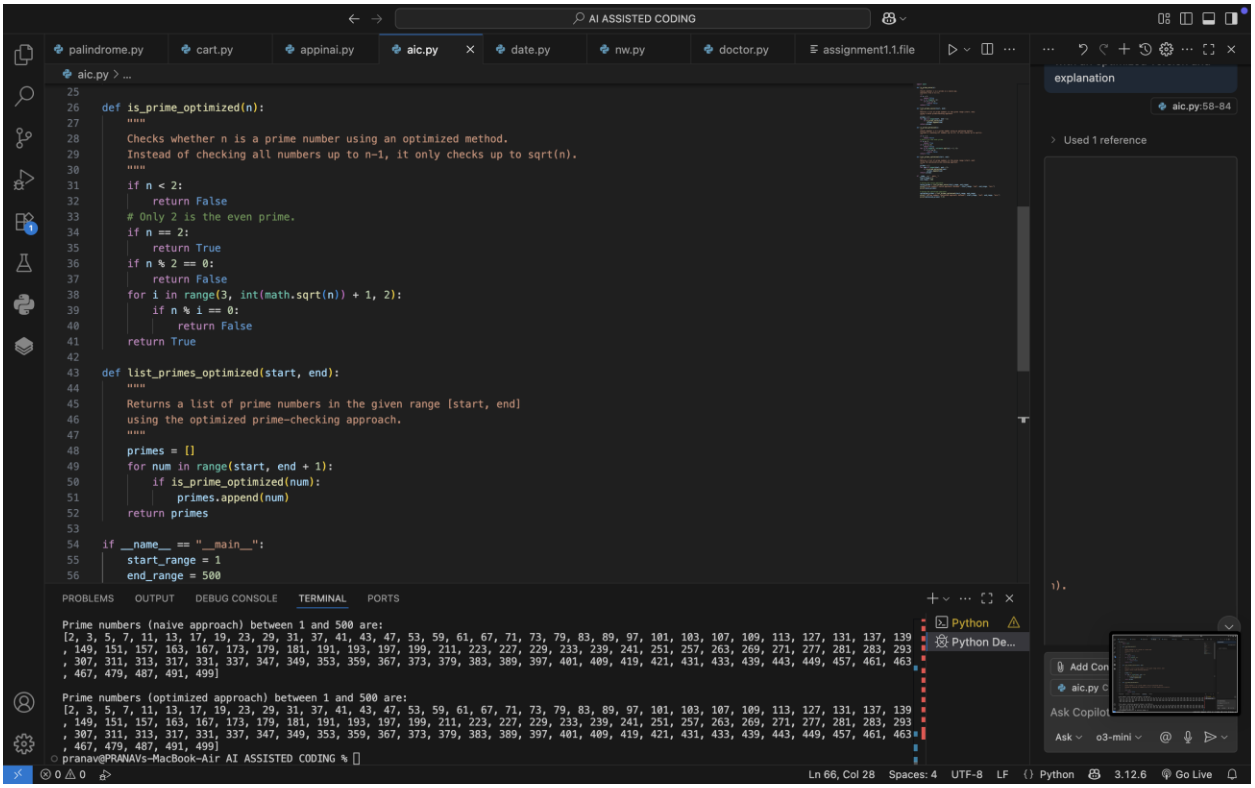
PROMPT USED :

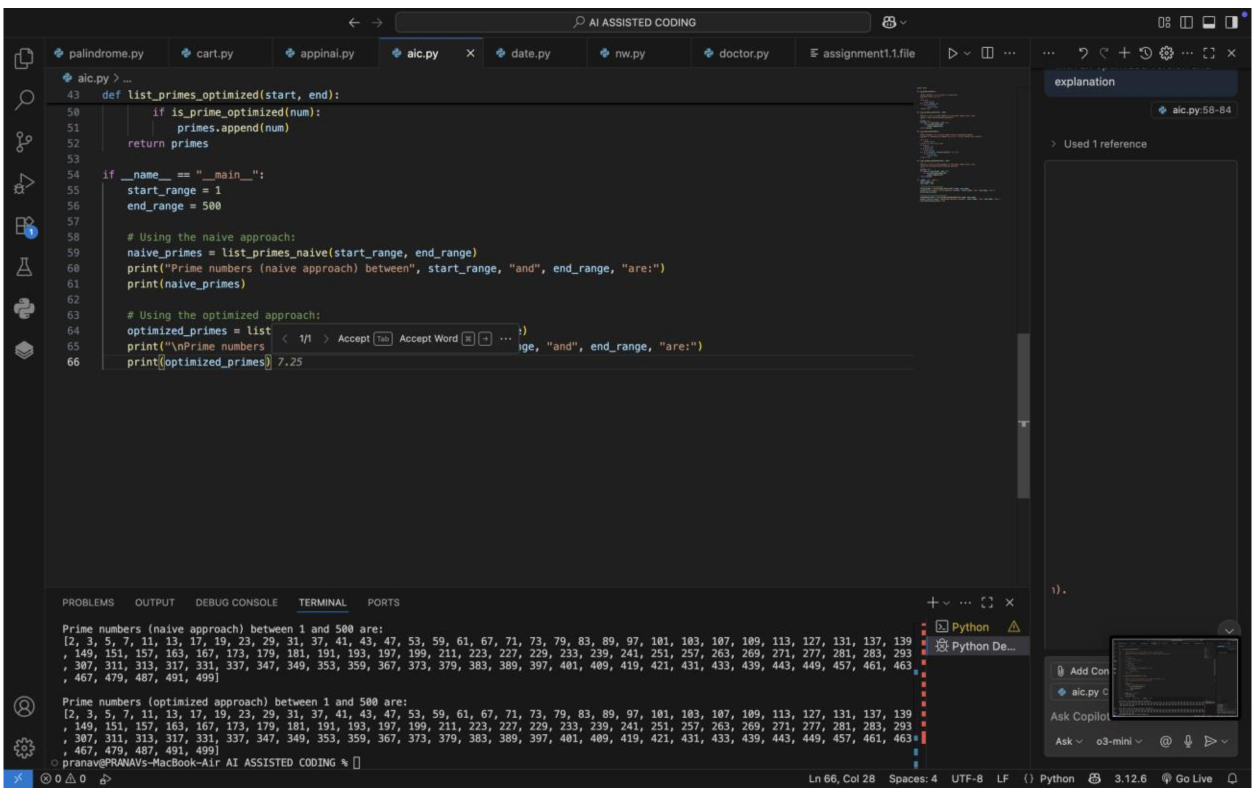
To Generate a function that displays all prime numbers within a user-specified range (e.g., 1 to 500)

To list all primes using a for loop.

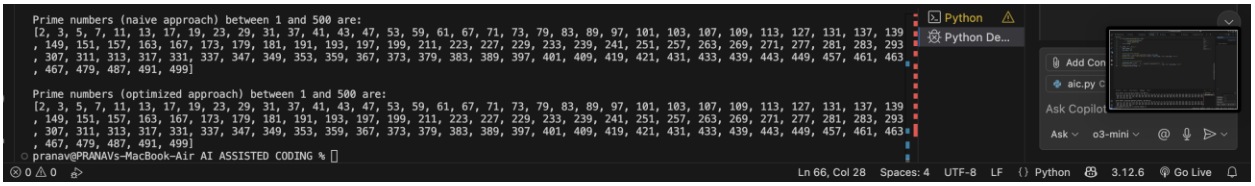
CODE :




AI ASSISTED CODING
palindrome.py
cart.py
appinai.py
aic.py
date.py
nw.py
doctor.py
assignment1.1.file
explanation
aic.py:58-84
Used 1 reference
def is_prime_optimized(n):
"""
Checks whether n is a prime number using an optimized method.
Instead of checking all numbers up to n-1, it only checks up to sqrt(n).
"""
if n < 2:
return False
# Only 2 is the even prime.
if n == 2:
return True
if n % 2 == 0:
return False
for i in range(3, int(math.sqrt(n)) + 1, 2):
if n % i == 0:
return False
return True
def list_primes_optimized(start, end):
"""
Returns a list of prime numbers in the given range [start, end]
using the optimized prime-checking approach.
"""
primes = []
for num in range(start, end + 1):
if is_prime_optimized(num):
primes.append(num)
return primes
if __name__ == "__main__":
start_range = 1
end_range = 500
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Prime numbers (naive approach) between 1 and 500 are:
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139
, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293
, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463
, 467, 479, 487, 491, 499]
Prime numbers (optimized approach) between 1 and 500 are:
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139
, 149, 151, 157, 163, 167, 173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293
, 307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367, 373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433, 439, 443, 449, 457, 461, 463
, 467, 479, 487, 491, 499]
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Python
Python De...
Add Con
Ask Copilot
Ask
o3-mini
Ln 66, Col 28 Spaces: 4 UTF-8 LF Python 3.12.6 Go Live


AI ASSISTED CODING
aic.py
def list_primes_optimized(start, end):
if is_prime_optimized(num):
primes.append(num)
return primes
if __name__ == "__main__":
start_range = 1
end_range = 500
# Using the naive approach:
naive_primes = list_primes_naive(start_range, end_range)
print("Prime numbers (naive approach) between", start_range, "and", end_range, "are:")
print(naive_primes)
# Using the optimized approach:
optimized_primes = list
print("\nPrime numbers
print(optimized_primes)
1/1 Accept Tab Accept Word
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Prime numbers (naive approach) between 1 and 500 are:
Prime numbers (optimized approach) between 1 and 500 are:
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Ln 66, Col 28 Spaces: 4 UTF-8 LF Python 3.12.6 Go Live

OUTPUT :



Task Description #5 (Classes – Library System)

• Task: Use AI to build a Library class with methods to add_book(), issue_book(), and display_books().

• Instructions:

o Generate Library class code using AI.

o Analyze if methods handle edge cases (e.g., issuing unavailable books).

o Ask AI to add comments and documentation.

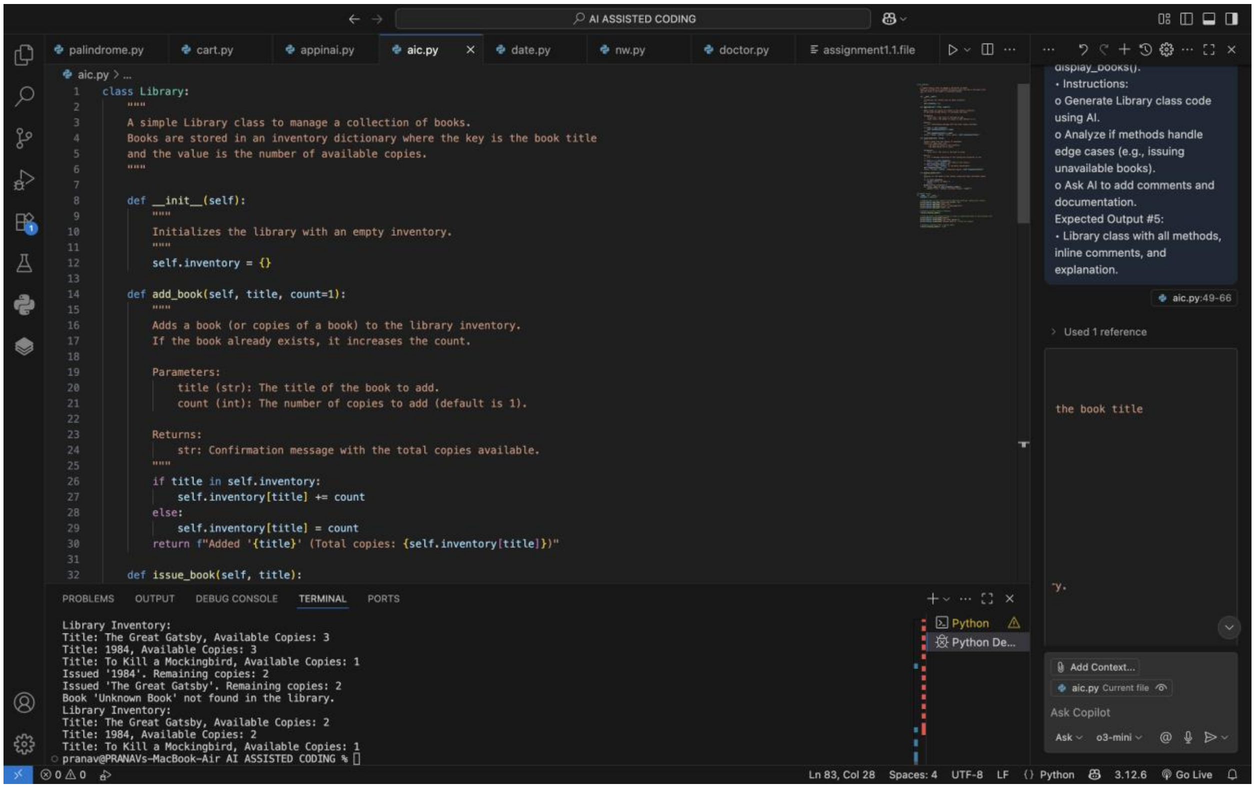
Expected Output #5:

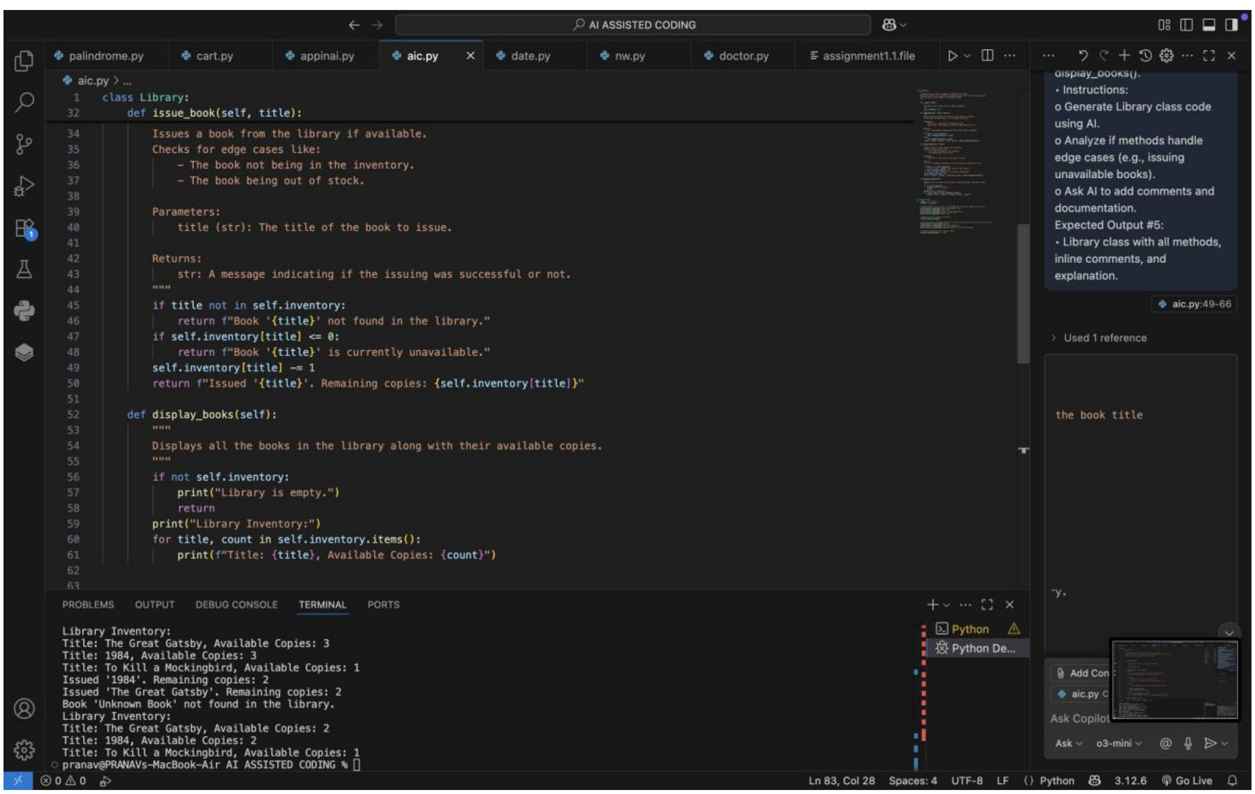
• Library class with all methods, inline comments, and explanation.

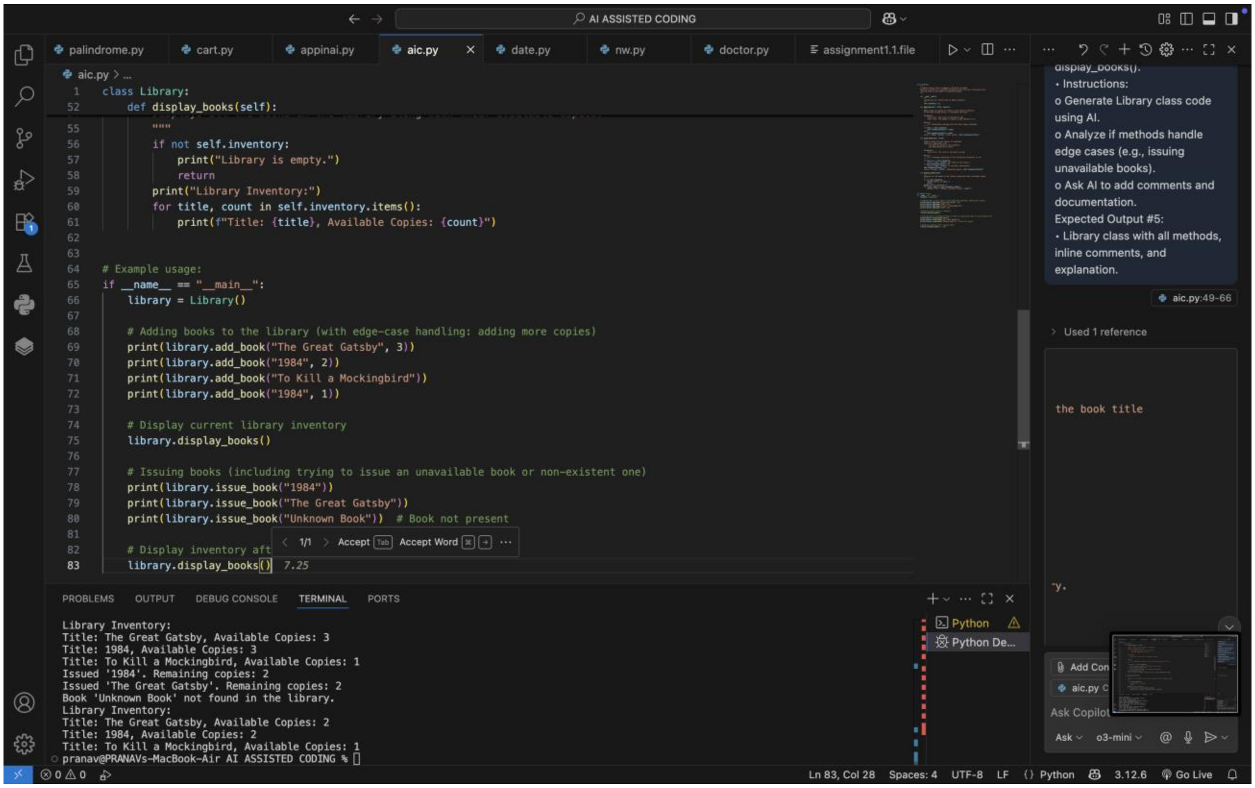
PROMPT :

To build a Library class with methods to
add_book(), issue_book(), and display_books()

CODE :


AI ASSISTED CODING
palindrome.py
cart.py
appinai.py
aic.py
date.py
nw.py
doctor.py
assignment1.1.file
aic.py > ...
class Library:
"""
A simple Library class to manage a collection of books.
Books are stored in an inventory dictionary where the key is the book title
and the value is the number of available copies.
"""
def __init__(self):
"""
Initializes the library with an empty inventory.
"""
self.inventory = {}
def add_book(self, title, count=1):
"""
Adds a book (or copies of a book) to the library inventory.
If the book already exists, it increases the count.
Parameters:
title (str): The title of the book to add.
count (int): The number of copies to add (default is 1).
Returns:
str: Confirmation message with the total copies available.
"""
if title in self.inventory:
self.inventory[title] += count
else:
self.inventory[title] = count
return f"Added '{title}' (Total copies: {self.inventory[title]})"
def issue_book(self, title):
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Library Inventory:
Title: The Great Gatsby, Available Copies: 3
Title: 1984, Available Copies: 3
Title: To Kill a Mockingbird, Available Copies: 1
Issued '1984'. Remaining copies: 2
Issued 'The Great Gatsby'. Remaining copies: 2
Book 'Unknown Book' not found in the library.
Library Inventory:
Title: The Great Gatsby, Available Copies: 2
Title: 1984, Available Copies: 2
Title: To Kill a Mockingbird, Available Copies: 1
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Python
Python De...
• Instructions:
o Generate Library class code using AI.
o Analyze if methods handle edge cases (e.g., issuing unavailable books).
o Ask AI to add comments and documentation.
Expected Output #5:
• Library class with all methods, inline comments, and explanation.
aic.py:49-66
Used 1 reference
the book title
Add Context...
aic.py Current file
Ask Copilot
Ask
o3-mini
Ln 83, Col 28
Spaces: 4
UTF-8
LF
Python
3.12.6
Go Live


AI ASSISTED CODING
aic.py > ...
class Library:
def issue_book(self, title):
Issues a book from the library if available.
Checks for edge cases like:
- The book not being in the inventory.
- The book being out of stock.
Parameters:
title (str): The title of the book to issue.
Returns:
str: A message indicating if the issuing was successful or not.
"""
if title not in self.inventory:
return f"Book '{title}' not found in the library."
if self.inventory[title] <= 0:
return f"Book '{title}' is currently unavailable."
self.inventory[title] -= 1
return f"Issued '{title}'. Remaining copies: {self.inventory[title]}"
def display_books(self):
"""
Displays all the books in the library along with their available copies.
"""
if not self.inventory:
print("Library is empty.")
return
print("Library Inventory:")
for title, count in self.inventory.items():
print(f"Title: {title}, Available Copies: {count}")
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Library Inventory:
Title: The Great Gatsby, Available Copies: 3
Title: 1984, Available Copies: 3
Title: To Kill a Mockingbird, Available Copies: 1
Issued '1984'. Remaining copies: 2
Issued 'The Great Gatsby'. Remaining copies: 2
Book 'Unknown Book' not found in the library.
Library Inventory:
Title: The Great Gatsby, Available Copies: 2
Title: 1984, Available Copies: 2
Title: To Kill a Mockingbird, Available Copies: 1
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
• Instructions:
o Generate Library class code using AI.
o Analyze if methods handle edge cases (e.g., issuing unavailable books).
o Ask AI to add comments and documentation.
Expected Output #5:
• Library class with all methods, inline comments, and explanation.
aic.py:49-66
Used 1 reference
the book title
Ask Copilot
Ln 83, Col 28
Spaces: 4
UTF-8
LF
Python
3.12.6
Go Live


AI ASSISTED CODING
palindrome.py
cart.py
appinai.py
aic.py
date.py
nw.py
doctor.py
assignment1.1.file
class Library:
def display_books(self):
if not self.inventory:
print("Library is empty.")
return
print("Library Inventory:")
for title, count in self.inventory.items():
print(f"Title: {title}, Available Copies: {count}")
# Example usage:
if __name__ == "__main__":
library = Library()
# Adding books to the library (with edge-case handling: adding more copies)
print(library.add_book("The Great Gatsby", 3))
print(library.add_book("1984", 2))
print(library.add_book("To Kill a Mockingbird"))
print(library.add_book("1984", 1))
# Display current library inventory
library.display_books()
# Issuing books (including trying to issue an unavailable book or non-existent one)
print(library.issue_book("1984"))
print(library.issue_book("The Great Gatsby"))
print(library.issue_book("Unknown Book")) # Book not present
# Display inventory aft
library.display_books()
Accept
Accept Word
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Library Inventory:
Title: The Great Gatsby, Available Copies: 3
Title: 1984, Available Copies: 3
Title: To Kill a Mockingbird, Available Copies: 1
Issued '1984'. Remaining copies: 2
Issued 'The Great Gatsby'. Remaining copies: 2
Book 'Unknown Book' not found in the library.
Library Inventory:
Title: The Great Gatsby, Available Copies: 2
Title: 1984, Available Copies: 2
Title: To Kill a Mockingbird, Available Copies: 1
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Python
Python De...
• Instructions:
o Generate Library class code using AI.
o Analyze if methods handle edge cases (e.g., issuing unavailable books).
o Ask AI to add comments and documentation.
Expected Output #5:
• Library class with all methods, inline comments, and explanation.
aic.py:49-66
Used 1 reference
the book title
Add Con
Ask Copilot
Ask
o3-mini
Ln 83, Col 28
Spaces: 4
UTF-8
LF
Python
3.12.6
Go Live

OUTPUT :


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Library Inventory:
Title: The Great Gatsby, Available Copies: 3
Title: 1984, Available Copies: 3
Title: To Kill a Mockingbird, Available Copies: 1
Issued '1984'. Remaining copies: 2
Issued 'The Great Gatsby'. Remaining copies: 2
Book 'Unknown Book' not found in the library.
Library Inventory:
Title: The Great Gatsby, Available Copies: 2
Title: 1984, Available Copies: 2
Title: To Kill a Mockingbird, Available Copies: 1
pranav@PRANAVs-MacBook-Air AI ASSISTED CODING %
Python
Python De...
Add Con
Ask Copilot
Ask
o3-mini
Ln 83, Col 28
Spaces: 4
UTF-8
LF
Python
3.12.6
Go Live